

计算机安全与保密技术课程成绩评价表

课程名称	计算机安全与保密技术	课程编号	08A65002
论文题目	基于 Socket 的 Web 代理服务器设计与实现		
学生姓名	谭坤	学号	23122781
所在院系	上海大学计算机工程与科学学院		

课程论文评阅记录

课程论文评价要素				
项目设计 35%	技术正确	内容完成	算法新颖	得分：
程序实现 35%	功能完成	运行稳定	设计合理	得分：
报告内容与格式 30%	报告清晰	见解独到	格式规范	得分：
报告值得肯定之处				
报告存在主要问题				
其他需要说明情况				

论文成绩：_____（百分制） 任课教师：_____

（课程论文占期末总评成绩的 60%）

评阅日期：_____

注：后接课程论文，格式参照课程论文模板或公开发表论文的风格。

项目仓库：github.com/terrytandragon-afk/web_proxy_final_lab

基于 Socket 的 Web 代理服务器设计与实现

谭坤 23122781

(上海大学计算机工程与科学学院, 上海 200444)

摘要: 为实现课程要求的 Web 请求转发、指定域名拦截和网页关键词过滤, 本文设计并实现了一套基于 Python Socket 的正向代理服务器。系统采用多线程处理客户端连接, 通过解析 HTTP 代理请求建立上游连接, 并按照客户端地址、认证状态、访问频率、请求方法、域名和 URL 的顺序执行访问控制; 对于明文 HTTP 文本响应, 代理在返回客户端前检查正文关键词。系统同时提供 HTTPS CONNECT 隧道、GET 缓存、动态规则管理、管理前端和日志审计功能。测试采用批量自动验证与浏览器手工验收相结合的方法, Web/代理功能批次 15 项、规则与运行模式批次 7 项均通过; 真实外网 URL 拦截和本地可控正文过滤均能够生成明确的 403 拦截页面。结果表明, 该系统完成了题目规定功能, 并具备规则热更新、运行统计和可追溯验收能力。

关键词: Web 代理; Socket; 访问控制; 关键词过滤; 日志审计; 动态规则

Design and Implementation of a Socket-Based Web Proxy Server

TAN KUN 23122781

School of Computer Engineering and Science, Shanghai University, Shanghai 200444, China

Abstract: This project designs and implements a forward Web proxy server using Python sockets. The proxy parses HTTP proxy requests, establishes upstream connections, forwards responses, and applies access-control policies based on client IP addresses, authentication, request rate, HTTP methods, domains, and URL keywords. For plain HTTP text responses, the proxy examines response bodies and replaces blocked pages with an explicit HTTP 403 notice. The system also provides HTTPS CONNECT tunneling, GET caching, runtime rule management, an administration dashboard, and auditable logs. Automated batch tests and browser-based manual tests were used for verification. All 15 Web/proxy tests and 7 rule/runtime-mode tests passed; a real public URL and a locally controlled response body were both blocked as expected. The results show that the system satisfies the course requirements and provides practical rule updates, statistics, and traceable acceptance evidence.

Key words: Web proxy; Socket; access control; keyword filtering; audit log; dynamic rule

目录

1	课题背景与功能分析	4
1.1	开发目的	4
1.2	功能需求分析	4
1.3	开发环境与技术路线	4
2	系统总体设计	4
2.1	总体架构	4
2.2	模块组成	5
2.3	数据结构与配置模型	5
2.4	请求处理流程	5
3	核心功能原理、设计与实现	6
3.1	HTTP 正向代理与请求转发原理	6
3.1.1	代理请求解析	7
3.1.2	请求改写与安全处理	7
3.1.3	上游连接与响应边界判定	7
3.2	访问控制策略链原理	7
3.2.1	有序短路策略	8
3.2.2	域名与 URL 规则设计	8
3.2.3	客户端入口控制	8
3.3	网页正文关键词过滤原理	8
3.3.1	过滤位置与资源类型判断	8
3.3.2	响应还原与字符解码	9
3.3.3	关键词匹配与替代响应	9
3.4	HTTPS CONNECT 隧道原理	9
3.4.1	隧道建立	9
3.4.2	加密字节双向转发	9
3.4.3	HTTPS 可见性边界	9
3.5	缓存、认证与限流设计	9
3.5.1	HTTP GET 缓存	9
3.5.2	代理认证	10
3.5.3	固定窗口限流	10
3.6	动态规则管理与并发一致性	10
3.6.1	统一操作入口	10
3.6.2	规则标准化与持久化	10
3.6.3	并发与关联状态一致性	11
3.7	日志审计与管理前端设计	11
3.7.1	事件模型与分类存储	11
3.7.2	结构化查询与证据隔离	11
3.7.3	管理前端展示	11
4	测试方案与验收结果	11
4.1	批量验证式测试	11
4.2	手工验收准备	13

4.3	普通转发与 HTTPS 隧道验收	13
4.4	真实网站域名拦截验收	13
4.5	真实网站 URL 关键词验收	14
4.6	本地可控网页正文关键词验收	15
4.7	缓存、认证与限流验收	16
4.8	日志、统计与规则变更验收	18
4.9	测试结果分析	19
5	项目开发总结	20
5.1	完成情况	20
5.2	存在问题与改进方向	20
6	心得体会	20
	参考文献	21
A	关键源代码	21
A.1	HTTP 请求解析、改写与转发	21
A.2	访问控制策略判定	22
A.3	HTTP 明文正文关键词过滤	22
A.4	HTTPS CONNECT 双向隧道	23
A.5	缓存、认证与固定时间窗口限流	24
A.6	动态规则与缓存一致性	25
A.7	日志分类写入与结构化解析	26

1 课题背景与功能分析

1.1 开发目的

Web 代理服务器位于客户端和目标 Web 服务器之间。客户端将访问请求发送给代理，代理解析请求中的目标地址，代替客户端连接目标服务器，再把响应转交给客户端。由于请求和响应均经过代理，代理能够在建立上游连接前判断目标是否允许访问，也能够收到明文响应后检查网页正文。该工作过程覆盖了 TCP 连接、HTTP 报文解析、访问控制、内容过滤和安全审计等网络安全基础知识。

本课题的开发目的包括三个方面。第一，完成题目规定的 HTTP 请求转发、指定域名拦截和网页关键词过滤功能。第二，通过 Socket 和 HTTP 协议解析理解正向代理的数据处理过程，并明确普通代理面对 HTTPS 加密流量时的可见性边界。第三，建立可管理、可验证和可审计的实验系统，使规则能够在运行期间修改，使每次放行与拦截都能够通过统计和日志进行说明。

1.2 功能需求分析

项目功能按照课程必做功能、系统管理功能和扩展访问控制功能进行划分。课程必做功能直接对应题目要求；系统管理功能用于解决规则如何修改、结果如何展示、行为如何追溯的问题；扩展访问控制功能用于体现代理服务器在实际网络安全场景中的使用方式。

表 1 Web 代理服务器功能需求分析

功能类别	功能说明
代理转发	接收客户端 HTTP 请求，连接目标服务器，将请求和响应完整转发。
域名与 URL 控制	支持完整域名、父域名后缀、显式通配符和 URL 关键词拦截。
正文关键词过滤	检查明文 HTTP 文本响应，命中关键词时返回代理生成的 403 页面。
客户端访问控制	支持 IPv4、IPv6 和 CIDR 网段的客户端黑名单与白名单。
运行策略	支持请求方法过滤、HTTP Basic 代理认证、固定时间窗口限流和 GET 缓存。
规则管理	支持前端、JSON API 和命令行三种管理方式，规则增删改查后立即生效。
展示与审计	展示统计、排行、缓存状态和规则变更，支持结构化日志查询与 CSV 导出。

1.3 开发环境与技术路线

系统运行于 Windows 环境，使用 Visual Studio Code 进行开发，主要编程语言为 Python 3。代理服务使用 socket 建立 TCP 监听与上游连接，使用 threading 为每个客户端连接提供独立处理线程，HTTPS CONNECT 隧道使用 select 实现双向字节转发。管理后端使用 Python 标准库中的 ThreadingHTTPServer 提供页面和 JSON API，管理前端使用 HTML、CSS 与 JavaScript 实现。

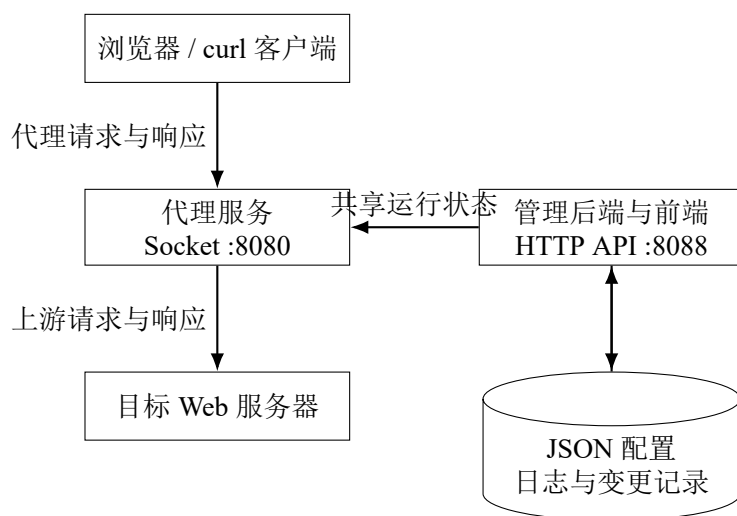
配置和规则使用 JSON 文件保存，规则变更使用 JSONL 逐行记录，访问事件写入结构化文本日志。客户端 IP 和 CIDR 使用 ipaddress 模块校验，域名通配符使用 fnmatch 匹配。该技术路线不依赖大型第三方框架，便于在课程报告中说明协议处理和策略判定的底层过程。

2 系统总体设计

2.1 总体架构

系统由代理服务、管理后端、管理前端、配置与日志存储、测试工具五部分组成。代理服务监听 127.0.0.1:8080，处理浏览器或命令行客户端发来的代理请求；管理后端监听 127.0.0.1:8088，向前端和命令行工具提供管理 API。两个服务共享运行状态，规则修改后能够立即影响后续代理请

求。



2.2 模块组成

代理核心模块负责监听端口、解析 HTTP 请求、执行策略链、转发普通 HTTP 请求和建立 HTTPS CONNECT 隧道。规则配置模块负责读取配置文件、定义规则组、校验并标准化规则值。日志审计模块负责写入访问、拦截和错误事件，并提供查询、清空和 CSV 导出。证据模块负责读取批量测试生成的独立日志，在管理前端中重建最近验收统计。管理前端负责规则编辑、运行设置、统计、日志和规则变更展示。

2.3 数据结构与配置模型

运行状态对象保存当前配置、统计计数、访问排行、关键词命中排行、缓存、限流桶和规则变更历史。共享状态由线程锁保护，避免多个客户端线程和管理线程同时修改数据时产生不一致。

列表型规则包括域名黑名单、域名白名单、客户端 IP 黑名单、客户端 IP 白名单、URL 关键词、正文关键词和禁止方法。运行设置包括黑白名单模式、缓存开关与容量、代理认证、限流上限和时间窗口。规则变更后，系统同时更新内存配置和 JSON 文件，使修改立即生效并在重启后继续保留。

2.4 请求处理流程

请求进入代理后按照固定顺序处理。客户端 IP 规则最先执行，使无权使用代理的主机无法消耗认证和限流资源；认证通过后执行访问频率检查；随后检查请求方法、域名和 URL。前置策略通过后，GET 请求查询缓存，普通 HTTP 请求连接目标服务器并检查响应正文，CONNECT 请求建立加密字节隧道。最后更新统计并写入日志。

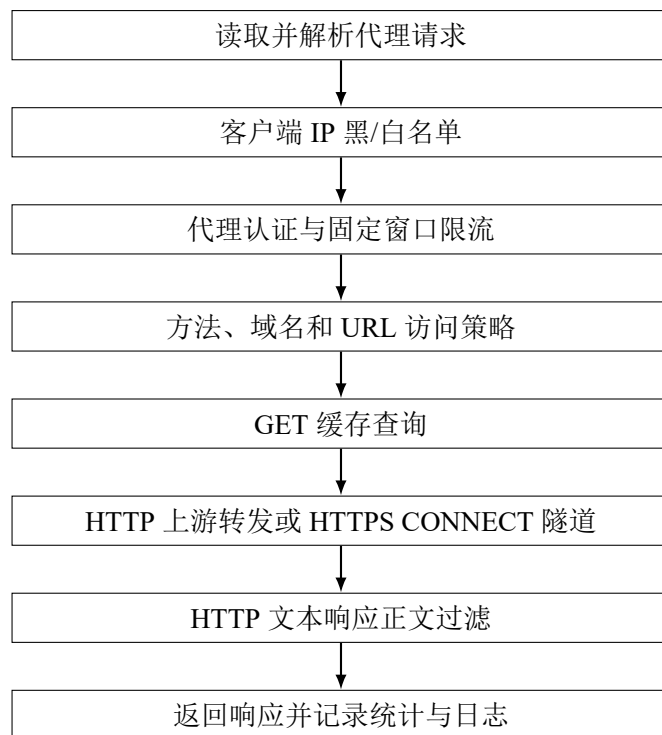


图 2 代理请求处理流程

3 核心功能原理、设计与实现

本系统的核心功能按照“解析请求、执行前置策略、访问目标服务器、检查返回内容、记录结果”的主线设计。实现时优先使用 Python 标准库提供的底层网络与数据处理能力：`socket` 负责 TCP 通信，`threading` 负责并发连接，`select` 负责 CONNECT 隧道双向转发，`gzip` 与 `zlib` 负责压缩正文检查，`json` 与文件日志负责配置持久化和审计。各功能共享同一请求信息对象和运行状态对象，使策略、缓存、日志及前端统计使用一致的数据来源。

3.1 HTTP 正向代理与请求转发原理

HTTP 正向代理模块负责完成客户端与目标服务器之间的通信闭环。该模块总体设计为四个阶段：接收并解析客户端代理请求、生成目标服务器能够理解的上游请求、通过 TCP Socket 收取完整响应、把响应交给正文过滤和客户端回传。代理不依赖现成代理框架，便于直接观察请求行、请求头和响应边界的处理过程。

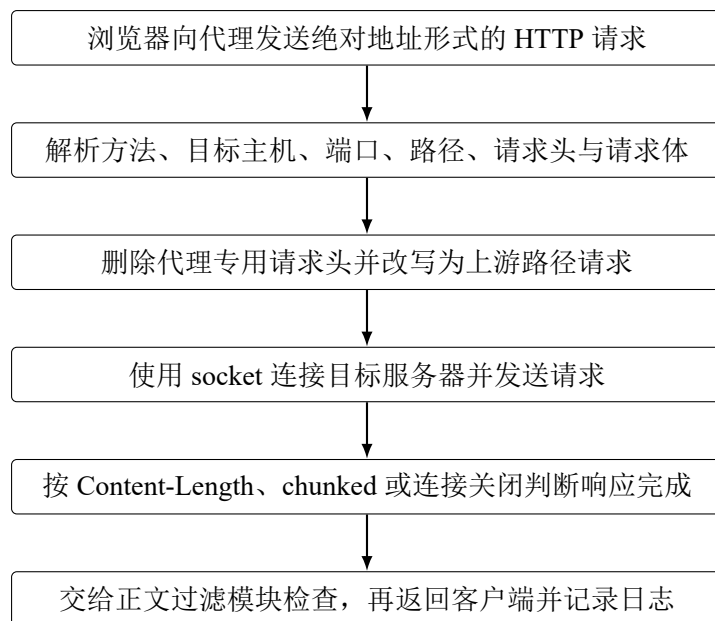


图 3 HTTP 正向代理请求转发流程

3.1.1 代理请求解析

浏览器直连网站时，请求行通常采用 `GET /path` 的路径形式；使用 HTTP 代理时，请求行采用携带完整目标地址的绝对形式。实现中先使用字符串分割请求行、请求头和请求体，再使用 `urlsplit` 解析绝对地址，从中取得目标主机、端口、路径和协议版本。若目标地址缺失或端口非法，代理在建立上游连接前终止处理，避免把不完整请求发送到网络。

解析结果统一保存为请求信息对象，后续域名策略、URL 关键词、缓存键和日志都读取该对象。这样可以避免每个功能重复解释原始报文，也使策略判断与转发使用同一目标信息。

3.1.2 请求改写与安全处理

目标 Web 服务器接收普通 HTTP 请求，因此代理需要把绝对地址请求行改写成路径形式，并根据解析结果重新生成 Host 头。转发前删除 `Proxy-Authorization`、`Proxy-Connection` 等只适用于客户端到代理之间的请求头，防止代理凭据泄露给目标网站。系统补充 `Connection: close` 和 `Accept-Encoding: identity`，前者请求服务器在响应结束后关闭连接，后者尽量获取可直接检查的未压缩正文。

3.1.3 上游连接与响应边界判定

代理使用 `socket.create_connection` 连接目标主机和端口，设置超时时间后发送改写请求，并按 HTTP 报文边界接收响应。响应声明 `Content-Length` 时，收到指定长度正文即可完成；使用 `chunked` 传输时，以 0 长度结束块和尾部空行作为结束；HEAD、204 和 304 等无正文响应在响应头接收完成后立即结束。只有响应没有长度信息时，才使用上游关闭连接作为结束标志。

该设计解决了部分真实网站在完整响应发送后仍保持 TCP 连接的问题。若仍单纯等待连接关闭，代理会在正文已经到齐的情况下误判超时，正文过滤也无法执行。目标连接失败时系统返回 502，确实未收到完整响应且等待超时时返回 504。请求解析、改写、HTTP 边界接收和上游转发实现见【A.1】。

3.2 访问控制策略链原理

访问控制模块采用前置检查与短路返回设计。每个策略函数只负责一种判断，并返回稳定的允许或拒绝结果；连接目标服务器之前依次执行这些函数，任一规则命中即停止后续处理。该设计把安全策略与网络转发解耦，新增规则组时只需在策略链中加入对应判断，同时日志能够精确记录拒绝发生的位置。

3.2.1 有序短路策略

访问控制采用有序、短路式策略链。客户端请求依次经过客户端地址策略、代理认证、访问频率、请求方法、白名单模式、域名黑名单和 URL 关键词判断。每项策略返回“允许”或“拒绝”以及稳定原因；任一策略拒绝后，代理立即生成响应并停止连接目标服务器。该顺序既减少无效上游连接，也能明确说明请求被哪一层拒绝。

3.2.2 域名与 URL 规则设计

域名规则先将目标主机转换为小写并移除末尾点，再支持完整域名、父域名后缀和显式通配符匹配。例如 `*.bing.com` 可以匹配 `www.bing.com` 和其他子域名。URL 关键词检查由主机、路径和查询参数组成的请求目标，仅适用于代理能够读取请求地址的明文 HTTP；它不检查目标服务器返回的正文。

黑名单模式下，命中禁止方法、域名或 URL 关键词即拒绝；白名单模式下，目标域名必须命中允许列表。策略结果使用 `domain_blacklist`、`domain_not_in_whitelist`、`url_keyword:...` 和 `method_blacklist` 等固定原因字段，拦截页面、日志、前端统计和测试都据此识别功能类别。

3.2.3 客户端入口控制

客户端地址策略使用套接字源地址，并通过 `ipaddress` 判断单个 IP 或 CIDR 网段；黑名单优先于白名单。认证与限流位于目标策略之前，使未认证请求和超额请求不会继续访问外部网络。策略链、域名通配符、URL 判断和客户端地址处理实现见【A.2】。

3.3 网页正文关键词过滤原理

正文过滤模块位于“上游响应接收”和“响应返回客户端”之间，总体流程为响应类型判断、传输与压缩还原、字符解码、关键词匹配和替代响应生成。该模块只处理代理能够读取的明文 HTTP 文本内容，并以本地可控测试页验证正文确实来自服务器响应而非请求 URL。

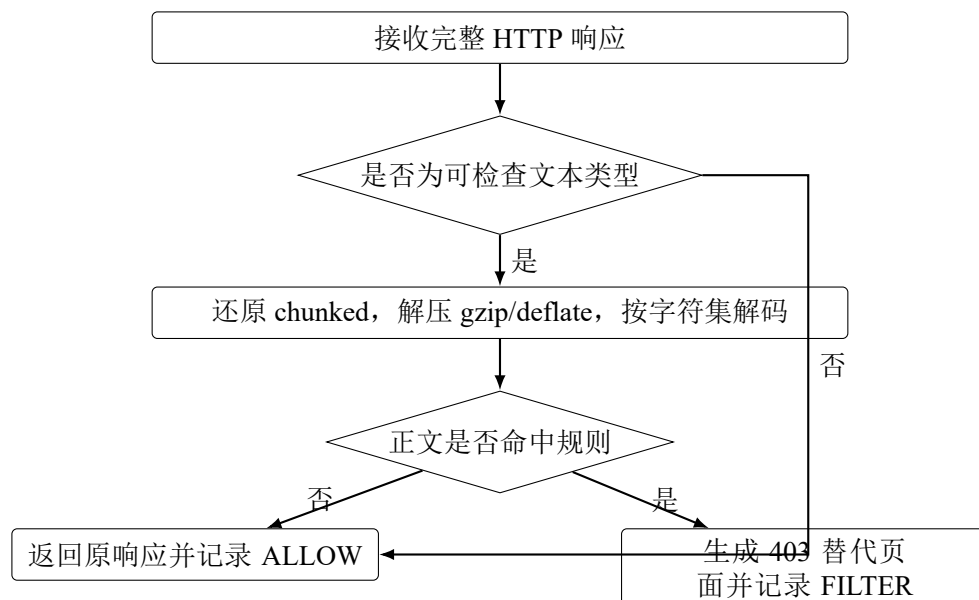


图 4 网页正文关键词过滤流程

3.3.1 过滤位置与资源类型判断

正文过滤位于上游响应完整接收之后、响应发送给客户端之前。系统先分离响应头和响应体，再读取 `Content-Type`、`Content-Encoding` 与 `Transfer-Encoding`。只有 HTML、纯文本、CSS、JavaScript 和 JSON 等文本资源进入检查流程；图片、音视频和其他二进制资源直接转发，避免将任意二进制字节错误解释成字符。

3.3.2 响应还原与字符解码

真实网站返回格式并不统一。对于 `chunked` 响应，系统先解析每个十六进制块长度并拼接有效正文，使跨数据块的关键词仍能命中；对于 `gzip` 和 `deflate` 压缩文本，系统先解压用于检查。字符解码优先使用响应头声明的字符集，随后尝试 UTF-8、GB18030 和 ISO-8859-1，从而覆盖常见中英文网页。

正文是否能够被检查还取决于上游响应能否正确结束。系统按 `Content-Length` 或 `chunked` 结束块及时完成接收，避免真实网站保持连接造成 504，从而保证已经到达的正文能够进入过滤流程。

3.3.3 关键词匹配与替代响应

正文解码后统一转换为小写，再与正文规则逐项比较，因此规则匹配不区分大小写。命中时，代理不再把原网页发送给客户端，而是生成状态码为 403 `Forbidden` 的替代页面；页面包含命中关键词、稳定原因 `content_keyword:...`、请求方法、目标地址和客户端地址。同时更新正文过滤统计并写入 `FILTER` 日志。

正文过滤仅适用于代理能够读取的明文 HTTP 响应。HTTPS 正文位于 TLS 加密隧道内，普通 `CONNECT` 代理无法读取。文本类型判断、分块还原、压缩解码、字符集处理和关键词替代实现见【A.3】。

3.4 HTTPS CONNECT 隧道原理

HTTPS `CONNECT` 模块采用“先检查目标、再建立透明隧道”的设计。由于 TLS 加密发生在浏览器与目标网站之间，代理只处理 `CONNECT` 请求中的目标域名和端口，并在允许访问后使用 `select` 转发两端加密字节。该模块能够完成 HTTPS 转发与域名控制，同时保留对加密内容不可见的安全边界。

3.4.1 隧道建立

浏览器访问 HTTPS 网站时，先向代理发送 `CONNECT host:443`。代理解析目标域名和端口，并在建立隧道前执行域名等前置策略。目标允许访问时，代理连接目标服务器并向浏览器返回 200 `Connection Established`；目标被禁止时，代理拒绝 `CONNECT`，浏览器通常显示隧道连接失败。

3.4.2 加密字节双向转发

隧道建立后，系统将客户端 `Socket` 和上游 `Socket` 设置为非阻塞状态，使用 `select` 同时监视两端。客户端有数据时转发给网站，网站有数据时转发给客户端；任一端关闭连接或长时间无活动时结束隧道。该模型避免为两个方向分别阻塞等待，并能统计隧道传输字节数。

3.4.3 HTTPS 可见性边界

`CONNECT` 隧道不解密 TLS。代理能够看到目标域名和端口，因此可以执行 HTTPS 域名拦截；代理看不到 HTTPS 内部路径、查询参数和正文，因此不能执行 HTTPS URL 路径或正文关键词过滤。实现这些能力需要中间人证书、客户端信任和私钥保护机制，不属于本实验范围。`CONNECT` 建立与双向转发实现见【A.4】。

3.5 缓存、认证与限流设计

缓存、认证和限流属于代理运行策略，分别解决重复请求性能、代理使用者身份和访问频率控制问题。三项功能都在运行状态对象中保存数据，并能够通过管理 API 或命令行即时调整。处理顺序为先认证、再限流、最后查询缓存，避免未授权或超额请求消耗上游资源。

3.5.1 HTTP GET 缓存

缓存键由目标主机、端口和完整路径组成，每个条目保存原始上游响应、状态码、创建时间和过期时刻。`GET` 请求到达时先检查缓存；未命中才连接目标服务器，命中则直接返回保存的响应。浏览器验收必须在同一代理进程、TTL 有效期内访问完全相同的 URL，并以日志中的 `CACHE\_HIT` 以及上游计数不再增长作为依据。

正文规则增删改或缓存参数变化时，系统清空旧缓存；缓存命中时仍按当前正文规则重新检查原始响应。这样可以避免先缓存允许页面、后新增正文规则时继续返回旧网页。

3.5.2 代理认证

代理认证使用 HTTP Basic 机制。请求未携带正确 Proxy-Authorization 时返回 407 和 Proxy-Authenticate 响应头，并记录 AUTH_REQUIRED。浏览器可能弹出认证对话框，也可能直接显示连接失败，因此验收以响应状态和审计日志为准。认证成功后才进入限流和目标策略，转发时删除代理认证头，保证目标网站不会收到代理账号密码。

3.5.3 固定窗口限流

限流为每个客户端 IP 保存当前窗口开始时间和已通过请求数。窗口内请求数达到上限后返回 429，并记录 count、limit 与 retry_after。认证挑战不消耗已认证请求额度，修改限流参数或执行重置命令时清空旧计数桶。缓存、代理认证和固定时间窗口限流实现见【A.5】。

3.6 动态规则管理与并发一致性

动态规则管理采用“统一 API、内存立即生效、文件持续保存、变更全程留痕”的总体设计。前端与命令行工具只是不同的操作入口，最终都调用相同后端方法。代理线程读取共享运行状态，管理线程负责修改状态，二者使用互斥锁保证并发一致性。

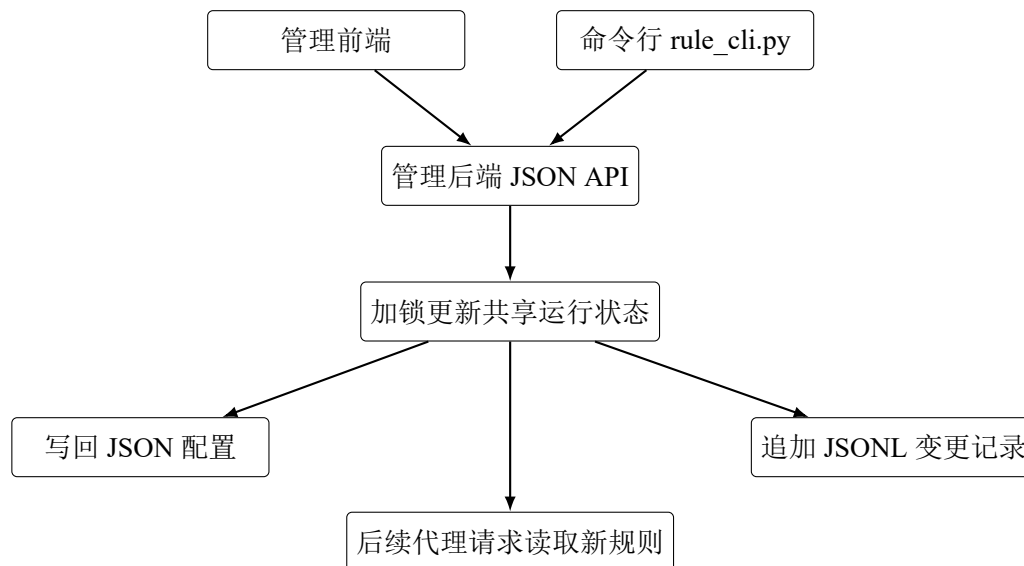


图 5 动态规则修改与立即生效流程

3.6.1 统一操作入口

规则可以通过管理前端、JSON API 或 rule_cli.py 命令行工具修改。前端和命令行最终都调用管理 API，API 再调用运行状态对象中的同一组新增、删除、修改、查询和整组替换方法。因此不同入口具有相同校验规则和立即生效语义。

3.6.2 规则标准化与持久化

后端先确认规则组是否允许编辑，再按类型标准化输入：域名和关键词去除多余空白，请求方法转换为大写，IP/CIDR 使用 ipaddress 校验。修改成功后同时更新内存配置和当前 JSON 配置文件，并向 JSONL 文件追加包含时间、操作、规则组、旧值、新值和保存结果的记录。新请求读取最新内存配置，代理无需重启。

3.6.3 并发与关联状态一致性

规则、统计、缓存和限流计数由运行状态对象统一管理，写操作在互斥锁内完成，避免代理线程和管理线程并发修改共享数据。正文规则变化时同步清空缓存，限流参数变化时同步重置计数桶，使关联状态与新配置保持一致。规则管理、缓存清理和变更记录实现见【A.6】。

3.7 日志审计与管理前端设计

日志与管理前端采用“代理产生事件、后端结构化查询、前端分类展示”的设计。代理在每个关键分支记录事件类型和原因，管理后端把文本日志解析为结构化字段，再向前端提供统计、筛选、详情和 CSV 导出接口。当前运行日志与批量验收证据分开保存，使现场操作和自动测试都能独立展示。

3.7.1 事件模型与分类存储

日志按照用途分为访问日志、拦截日志、错误日志和规则变更记录。访问事件包括 ALLOW、CONNECT、CACHE_HIT 等；拦截事件包括 BLOCK、FILTER、AUTH_REQUIRED 和 RATE_LIMIT。每条记录包含时间、客户端、请求方法、目标、路径、状态或稳定原因，便于从日志还原请求处理过程。

3.7.2 结构化查询与证据隔离

日志查询模块将文本行解析为字段，支持按日志类别、事件类型和全文关键词筛选，并能够导出 UTF-8 CSV。批量测试把 Web 行为证据与规则变更证据分别保存到独立目录；当前运行日志和批量证据使用不同查询入口，防止自动测试覆盖手工验收记录。

3.7.3 管理前端展示

运行状态对象维护总请求、放行、各类拦截、正文过滤、HTTPS 隧道、缓存、认证、限流和字节数统计。管理前端通过 JSON API 获取统计、规则、日志和变更历史；首页指标可跳转到对应日志查询页，规则变更页面可在“当前运行”和“批量验收”之间切换。日志写入、结构化查询和 CSV 导出实现见【A.7】。

4 测试方案与验收结果

4.1 批量验证式测试

项目首先使用批量自动验证检查完整功能，再进行浏览器和命令行手工验收。自动测试为每个模块创建独立端口、测试配置和上游目标，并断言 HTTP 状态、响应内容、配置保存、统计与日志。Web/代理批次与规则/运行模式批次分别使用 tests/evidence/web 和 tests/evidence/rules 保存证据，重复执行不会依赖正式配置中的已有规则，也不会相互覆盖日志。

测试批次	覆盖内容	结果
Web/代理功能	HTTP 转发、域名拦截、URL 与方法过滤、正文过滤、CONNECT、缓存、前端、认证、限流、浏览器代理和验收证据	15/15
规则与运行模式	管理 API、白名单模式、规则增删改查、CLI、运行设置、客户端 IP/CIDR 与规则变更证据	7/7

批量验证命令如下：

```
python tests\run_all_smoke.py
```

测试通过时输出 all test batches passed。管理前端选择“最近验收”后，能够从持久化证据重建总请求、总拦截、各类统计、日志和规则变更记录。

```
PASSED batch: web/proxy features
Passed tests: 15/15
Web evidence: tests/evidence/web/proxy.log and tests/evidence/web/blocked.log
Normal dashboard: restart it, open http://127.0.0.1:8088/, and use Recent Acceptance
PS E:\eve_jump\web_proxy_final_lab>
```

图 6 Web/代理功能批量验证结果

```
PASSED batch: rule groups and runtime modes
Passed tests: 7/7
Rule evidence: tests/evidence/rules/changes.jsonl
Normal dashboard: open http://127.0.0.1:8088/ and use Rule Change Overview
PS E:\eve_jump\web_proxy_final_lab>
```

图 7 规则与运行模式批量验证结果

图中两个批次分别显示 15/15 和 7/7，并最终输出 `all test batches passed`。该结果说明代理转发、访问控制、正文过滤、CONNECT、缓存、认证、限流以及规则运行时修改均通过断言；批次使用独立配置和端口，多次执行不会依赖正式配置中的规则状态，因此结果具有可重复性。



图 8 Web 批量验收结果在管理首页中的展示



图 9 规则批量验收变更总览

管理首页能够读取批量测试保存的证据并恢复总请求、总拦截和分类统计，规则变更总览能够展示新增、删除、更新和运行设置变化。两张图说明自动测试不仅判断命令执行成功，还留下了能够由管理前端查询的行为证据，测试结果与展示层数据保持一致。

4.2 手工验收准备

手工验收用于观察浏览器页面、命令行状态码、管理前端统计和日志之间是否一致。域名和 URL 规则使用真实网站验证，正文过滤使用项目自带的可控 HTTP 测试页验证，避免公网网站跳转、压缩方式和网络状态变化影响实验结果。验收前启动代理：

```
python src\proxy.py --config config.example.json
```

正文过滤与缓存验收需要在另一个终端启动本地测试服务器：

```
python tests\manual_demo_server.py
```

验收浏览器可使用系统代理，也可以使用项目提供的独立配置。独立浏览器关闭自动 HTTPS 升级并降低浏览器缓存干扰：

```
powershell -ExecutionPolicy Bypass -File tools\start_proxy_browser.ps1
```

进行普通转发、域名、URL 和正文验收前，先关闭认证和限流干扰并切换为黑名单模式：

```
python tools\rule_cli.py set mode blacklist
python tools\rule_cli.py set proxy_auth_enabled false
python tools\rule_cli.py set rate_limit_enabled false
python tools\rule_cli.py rate-reset
```

4.3 普通转发与 HTTPS 隧道验收

首先在未添加 Bing 域名规则时访问 <https://www.bing.com/>。浏览器向代理发送 CONNECT 请求，代理建立到目标网站的 TCP 连接并双向转发 TLS 字节流。页面正常显示，同时日志产生 CONNECT 记录，说明代理具备真实网站访问转发能力。

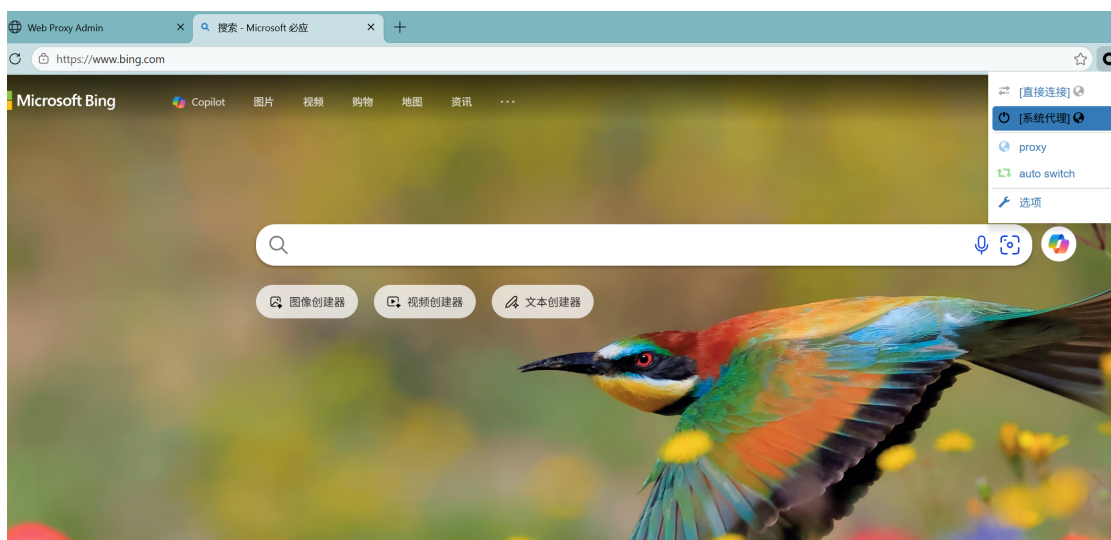


图 10 真实 HTTPS 网站通过代理正常访问

图中 Bing 页面能够正常加载，说明浏览器已经通过代理建立 CONNECT 隧道，代理能够连接真实目标服务器并双向转发 TLS 加密字节。该结果证明基础转发链路正常，也为后续添加域名黑名单后的对照实验提供了基准。

4.4 真实网站域名拦截验收

添加 Bing 子域名黑名单并访问真实 HTTPS 网站：

```
python tools\rule_cli.py add blocked_domains *.bing.com
```

在 Edge 中访问 <https://www.bing.com/>。浏览器会向代理发送 CONNECT www.bing.com:443，域名命中后代理拒绝建立 TLS 隧道。CONNECT 的 403 响应属于隧道协商失败，Edge 通常显示 ERR_TUNNEL_CONNECTION_FAILED，不会把代理返回的 HTML 403 正文渲染成网页。管理日志中的 BLOCK reason=domain_blacklist 用于确认具体原因。验收结束后删除规则：

```
python tools\rule_cli.py delete blocked_domains *.bing.com
```

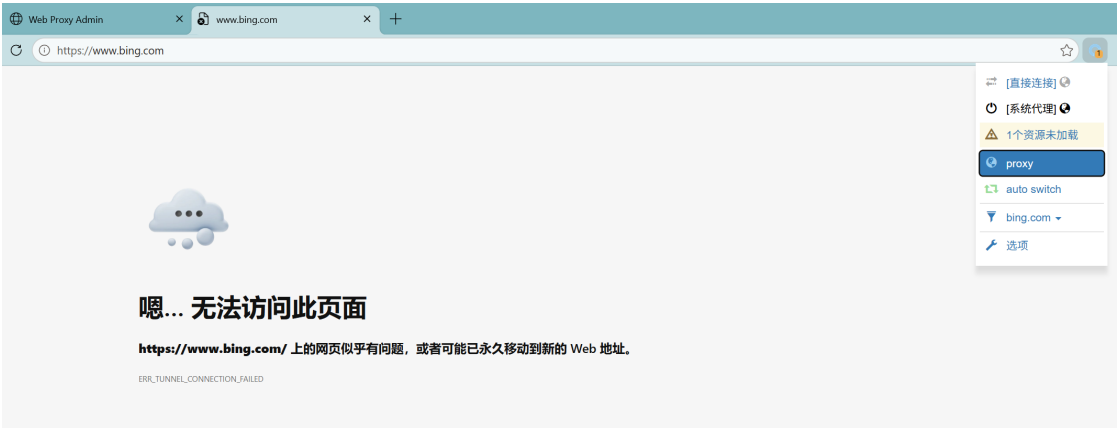


图 11 真实 HTTPS 网站域名拦截结果

总拦截日志

按日志类型、事件与关键字查询代理审计记录

返回管理平台

日志类型: 拦截日志 事件类型, 逗号分隔: 例如 BLOCK,FILTER 全文查询: 例如 domain_blacklist 或目标域名 最大结果数: 200 查询 导出 CSV

数据来源: 当前运行 日志类型: blocked 原始记录: 1182 匹配记录: 1182 事件条件: 全部 全文条件: 无

时间	事件	客户端	目标	详细信息
2026-06-13 11:08:31	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist

图 12 真实 HTTPS 域名拦截日志

浏览器在添加规则后显示隧道连接失败，而拦截日志明确记录目标主机为 www.bing.com、原因为 domain_blacklist。结合添加规则前能够正常访问的对照结果，可以确认失败由代理域名策略主动拒绝 CONNECT 引起，并非网站或网络临时不可用。该结果同时说明 HTTPS 场景下代理能够依据 CONNECT 目标域名实施控制，但浏览器不会渲染代理的普通 403 页面。

4.5 真实网站 URL 关键词验收

URL 关键词验收使用真实明文 HTTP 地址 <http://httpforever.com/url-filter-demo>。该路径包含稳定关键词 url-filter-demo，代理在连接目标网站之前即可完成判断：

```
python tools\rule_cli.py delete blocked_domains httpforever.com
python tools\rule_cli.py add blocked_url_keywords url-filter-demo
```

在专用 Edge 中输入完整地址 <http://httpforever.com/url-filter-demo>。预期显示代理生成的 403 页面，详细信息中的拦截原因为 url_keyword:url-filter-demo。验收结束后删除规则：

```
python tools\rule_cli.py delete blocked_url_keywords url-filter-demo
```



图 13 真实网站 URL 关键词拦截结果

图中地址栏保留了包含 `url-filter-demo` 的完整 HTTP 地址，页面被替换为代理生成的 403 提示，详细信息显示 `url\_keyword:url-filter-demo`。该结果说明代理能够在连接目标服务器之前读取明文 HTTP 请求路径并完成 URL 关键词判断；命中规则后没有继续访问目标页面，拦截原因能够直接反馈给使用者。

4.6 本地可控网页正文关键词验收

正文关键词验收使用项目测试服务器提供的地址 `http://127.0.0.1.nip.io:9000/content-test.html`。该 URL 本身不包含关键词 `forbidden`，关键词只存在于服务器返回的 HTML 正文中，因此能够区分 URL 拦截和正文过滤。使用 `127.0.0.1.nip.io` 访问本机服务，可以避免 Chromium 默认绕过本机代理。验收前清空代理缓存并添加正文规则：

```
python tools\rule_cli.py cache-clear
python tools\rule_cli.py delete blocked_url_keywords forbidden
python tools\rule_cli.py add blocked_content_keywords forbidden
```

在专用 Edge 中输入上述完整地址。预期原网页正文被代理生成的 403 页面替代，详细信息显示命中规则 `forbidden`，原因类型为正文关键词拦截。验收结束后删除规则：

```
python tools\rule_cli.py delete blocked_content_keywords forbidden
```

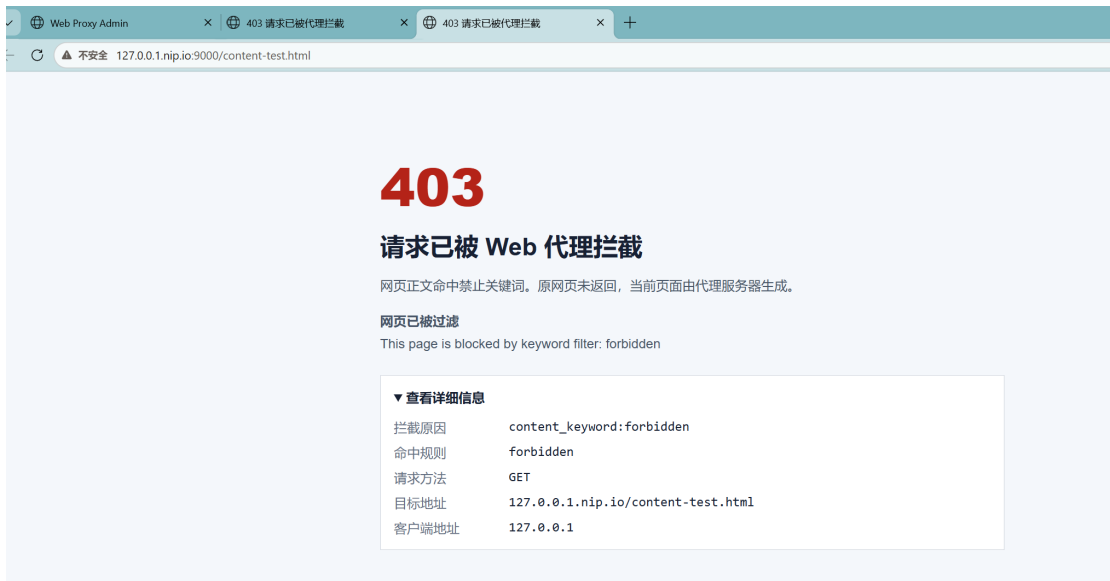


图 14 本地可控网页正文关键词过滤结果

图中目标地址为 127.0.0.1.nip.io/content-test.html，地址本身不含 forbidden，但代理页面显示命中规则和原因为 content_keyword:forbidden。这说明请求已通过 URL 前置策略并到达测试服务器，代理在收到响应正文后才发现关键词，随后丢弃原网页并返回 403 替代页面。该结果直接证明了正文过滤模块的处理位置和功能有效性。

4.7 缓存、认证与限流验收

缓存验收先启用缓存并清空旧条目，随后在 TTL 有效期内连续访问完全相同的 HTTP GET 地址。第一次日志应为 CACHE_MISS 与 ALLOW，第二次应为 CACHE_HIT，测试页中的 upstream-hit 保持不变。浏览器访问本地测试页时使用 127.0.0.1.nip.io，避免 Chromium 默认绕过本机代理；正文规则发生变化时，系统自动清空旧缓存，防止旧网页绕过新规则。

```
python tools\rule_cli.py set cache_enabled true
python tools\rule_cli.py cache-clear
```

总请求日志

按日志类型、事件与关键字查询代理审计记录

返回管理平台

日志类型

事件类型, 逗号分隔

全文查询

最大结果数

访问日志

例如 BLOCK,FILTER

例如 domain_blacklist 或目标域名

200

查询

导出 CSV

数据来源: 当前运行

日志类型: proxy

原始记录: 13096

匹配记录: 13096

事件条件: 全部

全文条件: 无

时间	事件	客户端	目标	详细信息
2026-06-13 11:20:59	CONNECT	127.0.0.1	cn.bing.com	client=127.0.0.1 host=cn.bing.com port=443 bytes=199
2026-06-13 11:20:59	ERROR	127.0.0.1	client-side-detection.api.cx.metamask.io client-side-detection.api.cx.metamask.io:443	client=127.0.0.1 host=client-side-detection.api.cx.metamask.io path=client-side-detection.api.cx.metamask.io:443 status=504
2026-06-13 11:20:55	CACHE_HIT	127.0.0.1	127.0.0.1/cache.txt	client=127.0.0.1 method=GET host=127.0.0.1 path=/cache.txt status=200 bytes=200
2026-06-13 11:20:54	ALLOW	127.0.0.1	127.0.0.1/cache.txt	client=127.0.0.1 method=GET host=127.0.0.1 path=/cache.txt status=200 bytes=200
2026-06-13 11:20:54	CACHE_MISS	127.0.0.1	127.0.0.1/cache.txt	client=127.0.0.1 method=GET host=127.0.0.1 path=/cache.txt

图 15 HTTP GET 缓存验收日志

缓存日志中，同一目标第一次访问产生 CACHE_MISS 和 ALLOW，第二次产生 CACHE_HIT；在 TTL 有效期内，测试页的 upstream-hit 不再增加。该结果说明第二次响应由代理内存缓存直接提供，没有再次连接上游测试服务器。若浏览器每次刷新都使 upstream-hit 增加，则表示请求未命中代理缓存，不能作为缓存成功的证据。

代理认证验收启用 **Basic** 认证。无凭据或错误凭据请求在协议层返回 **407**，浏览器可能弹出认证对话框，也可能直接显示连接失败，不保证显示代理生成的 **407** 页面。管理日志中的 **AUTH_REQUIRED** 用于确认认证拒绝，正确凭据请求能够继续访问，且 **Proxy-Authorization** 不转发给目标网站。

2026-06-13 11:24:37	CONNECT	127.0.0.1	www.bing.com	client=127.0.0.1 host=www.bing.com port=443 bytes=1448
2026-06-13 11:24:35	CONNECT	127.0.0.1	edge.microsoft.com	client=127.0.0.1 host=edge.microsoft.com port=443 bytes=9568
2026-06-13 11:24:31	CONNECT	127.0.0.1	login.live.com	client=127.0.0.1 host=login.live.com port=443 bytes=20449
2026-06-13 11:24:23	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:19	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:17	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:17	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:14	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:14	BLOCK	127.0.0.1	www.bing.com www.bing.com:443	client=127.0.0.1 method=CONNECT host=www.bing.com path=www.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:14	BLOCK	127.0.0.1	www.bing.com www.bing.com:443	client=127.0.0.1 method=CONNECT host=www.bing.com path=www.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:13	BLOCK	127.0.0.1	cn.bing.com cn.bing.com:443	client=127.0.0.1 method=CONNECT host=cn.bing.com path=cn.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:13	BLOCK	127.0.0.1	www.bing.com www.bing.com:443	client=127.0.0.1 method=CONNECT host=www.bing.com path=www.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:13	BLOCK	127.0.0.1	www.bing.com www.bing.com:443	client=127.0.0.1 method=CONNECT host=www.bing.com path=www.bing.com:443 reason=domain_blacklist
2026-06-13 11:24:04	AUTH_REQUIRED	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com
2026-06-13 11:24:04	AUTH_REQUIRED	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com
2026-06-13 11:24:03	AUTH_REQUIRED	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com
2026-06-13 11:23:59	AUTH_REQUIRED	127.0.0.1	api-shoulei-ssl.xunlei.com	client=127.0.0.1 method=CONNECT host=api-shoulei-ssl.xunlei.com

图 16 代理认证验收日志

认证日志中的 **AUTH_REQUIRED** 表明代理在访问目标网站之前识别出缺失或错误凭据并终止请求；随后出现的正常 **CONNECT** 记录说明关闭认证或提供正确凭据后请求能够继续。浏览器是否显示 **407** 页面属于客户端表现，日志和协议响应能够证明认证模块已经执行。

限流验收将窗口内请求上限设为 1，并在测试前重置计数。第一次请求记录 **RATE_ALLOW**，第二次请求返回 **429** 并记录 **RATE_LIMIT**。浏览器会产生附加资源请求，因此浏览器页面可能在主请求放行后很快出现限流失败；命令行连续请求能更准确展示第一次放行、第二次拒绝。



图 17 浏览器访问频率限制结果

限流拦截日志

按日志类型、事件与关键字查询代理审计记录

返回管理平台

日志类型	事件类型，逗号分隔	全文查询	最大结果数	
拦截日志	RATE_LIMIT	例如 domain_blacklist 或目标域名	200	查询 导出 CSV
数据来源: 当前运行 日志类型: blocked 原始记录: 1097 匹配记录: 521 事件条件: RATE_LIMIT 全文条件: 无				
时间	事件	客户端	目标	详细信息
2026-06-12 18:13:51	RATE_LIMIT	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com count=20 limit=20 retry_after=3
2026-06-12 18:13:50	RATE_LIMIT	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com count=20 limit=20 retry_after=3
2026-06-12 18:13:50	RATE_LIMIT	127.0.0.1	client-side-detection.api.cx.metamask.io	client=127.0.0.1 method=CONNECT host=client-side-detection.api.cx.metamask.io count=20 limit=20 retry_after=3
2026-06-12 18:13:49	RATE_LIMIT	127.0.0.1	cn.bing.com	client=127.0.0.1 method=CONNECT host=cn.bing.com count=20 limit=20 retry_after=4

图 18 访问频率限制日志

浏览器页面显示访问失败，日志同时给出 `RATE\_LIMIT`、当前计数、限制值和剩余等待时间。两项证据说明代理按照客户端 IP 维护固定时间窗口，并在额度耗尽后于访问目标网站之前拒绝后续请求。浏览器会并发请求页面资源，因此限流演示应结合日志判断具体被拒绝的请求。

4.8 日志、统计与规则变更验收

完成真实网站和运行设置验收后，管理首页“当前运行”数据源应显示本次总请求、总拦截和各分类统计。日志详情页支持事件与全文查询，规则变更总览默认读取当前运行历史；切换到“批量验收”后读取隔离的规则测试证据。

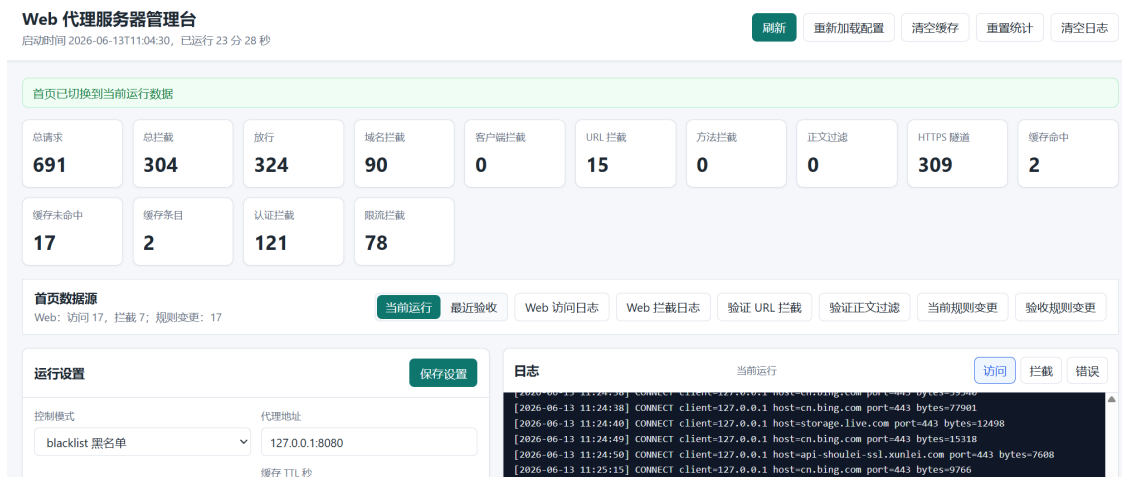


图 19 真实网站拦截日志查询结果

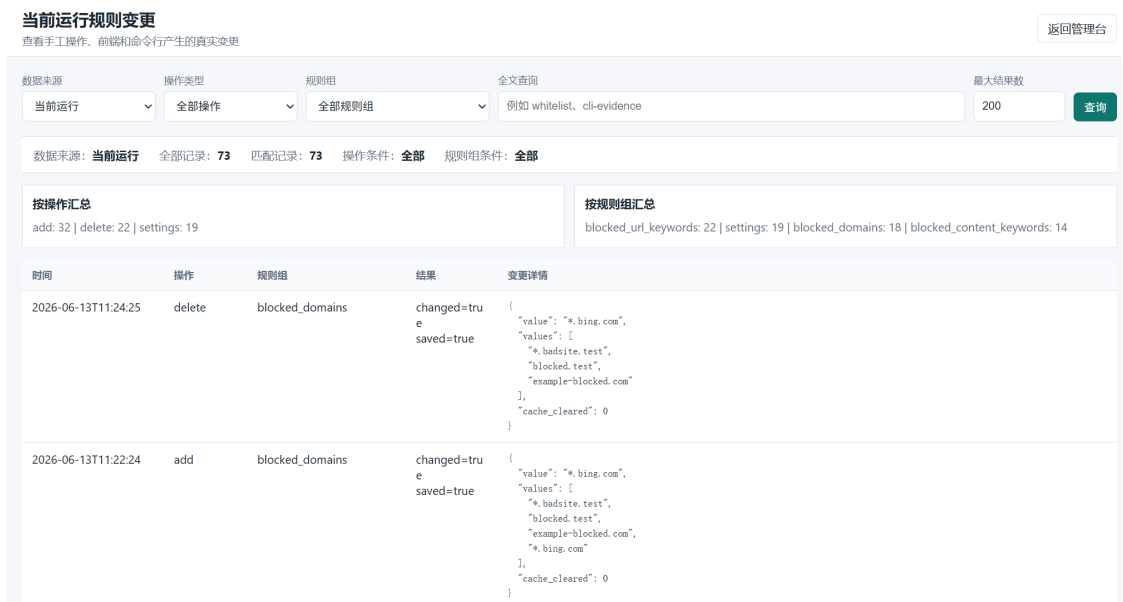


图 20 当前运行规则变更记录

拦截日志详情页能够按事件类型和关键词查询前述域名、URL、正文、认证与限流事件；规则变更总览能够显示手工验收过程中执行的新增、删除和设置操作。该结果说明系统不仅执行访问控制，还能够将请求处理结果与规则修改过程分别留痕，为故障定位和实验复核提供完整依据。

4.9 测试结果分析

两组批量测试最终分别通过 15/15 和 7/7，说明代理转发、访问控制、正文过滤、CONNECT、缓存、认证、限流、客户端地址策略、动态规则管理、前端统计和审计证据能够在独立环境中重复执行。浏览器端回归测试曾因统计字段名称写错而在完成 13 项后失败，修正为实际字段 `filtered_keyword` 后完整批次通过；该过程说明测试脚本本身也需要接受回归检查。

手工验收体现了 HTTP 与 HTTPS 的可见性差异。明文 HTTP URL 规则能够在建立上游连接前返回带原因的 403 页面，本地可控正文规则能够在接收响应后替换原网页；HTTPS 域名拦截发生在 CONNECT 阶段，Edge 将其显示为隧道连接失败，具体拒绝原因需要结合代理日志确认。每项手工验收均同时观察浏览器结果和日志记录，能够排除单纯页面加载失败造成的误判。

正文过滤故障排查发现，响应接收不能只依赖目标服务器关闭 TCP 连接。修复后，代理按照

Content-Length、chunked 结束块和无正文状态判断响应完成,并增加保持连接、gzip、deflate、chunked 与 GB18030 测试;正文规则变化仍会清空旧缓存,缓存命中也会按照当前规则重新检查。本地测试页提供稳定正文和明确关键词,使现场验收不再受公网网站行为变化影响。限流日志表明浏览器后台连接会共同消耗同一客户端 IP 的额度,因此现场演示其他模块前应关闭限流,单独验收限流时再启用并重置计数。

5 项目开发总结

5.1 完成情况

本项目完成了题目规定的 Web 请求转发、指定域名拦截和网页关键词过滤功能。代理能够解析 HTTP 代理请求并连接目标服务器,能够在连接目标服务器之前执行域名、URL、方法和客户端地址策略,也能够收到明文 HTTP 响应后执行正文关键词过滤。

在基础功能之上,项目实现了 HTTPS CONNECT、GET 缓存、代理认证、访问频率限制、黑白名单模式、动态规则管理、管理前端、统计与日志审计。规则可以通过前端、API 和命令行修改,并立即作用于后续请求。自动测试与浏览器手工验收能够从响应、日志、统计和规则变更多个角度说明功能结果。

5.2 存在问题与改进方向

当前代理不解密 HTTPS 流量,因此无法检查 HTTPS 路径和正文。正文过滤已经支持 UTF-8、GB18030、chunked、gzip 和 deflate 文本响应;Brotli 等其他压缩格式与大文件流式过滤仍可继续完善。缓存和统计保存在进程内存中,重启后会清空;日志使用文件保存,在数据量较大时查询效率有限。

后续可以增加管理端登录、规则快照与回滚、SQLite 审计存储、定时规则和更完整的 HTTP 流式处理。HTTPS 内容检查需要在受控实验环境中设计证书生成、客户端信任和私钥保护机制,并明确其安全与合规边界。

6 心得体会

本项目的主要收获是将 HTTP 协议、TCP Socket 和访问控制策略结合成一个可以实际运行的系统。实现代理转发后,可以直接观察浏览器请求经过代理、被改写、转发并返回的过程;实现域名与正文过滤后,可以区分“请求发送前拒绝”和“收到响应后过滤”两种处理位置。

开发过程中,对协议边界的理解比单纯完成代码更加重要。普通 HTTP 请求的路径和正文对代理可见,因此可以进行 URL 和内容检查;HTTPS 经过 CONNECT 建立 TLS 隧道后,代理只能依据目标域名进行控制。浏览器显示的隧道失败、认证提示或页面加载失败不能单独说明具体原因,必须结合代理记录的稳定原因字段进行判断。这个过程加深了我对“网络现象”和“程序内部处理结果”之间关系的理解。

项目调试也让我认识到真实网络环境会给实验带来许多额外变量。浏览器可能绕过本机代理、缓存页面、并发请求附加资源,公网网站可能跳转、压缩正文或保持 TCP 连接。正文过滤曾因响应已经接收完成但服务器未立即关闭连接而超时,最终通过按照 Content-Length 和 chunked 边界判断响应完整性解决。缓存验收则需要同时观察 CACHE_HIT 日志和上游访问计数,才能确认响应确实来自代理缓存。这些问题促使我把每项功能设计成可独立复现、可从日志解释的实验模块。

在工程组织方面,项目从单纯命令行代理逐步扩展为包含管理后端、前端页面、规则命令行工具、结构化日志和批量测试的完整系统。规则修改通过统一 API 进入共享状态,并同时保存到配置文件和变更记录;批量测试使用独立证据目录,避免影响手工验收数据。通过这一过程,我体会到完整项目不仅需要实现功能,还需要考虑如何配置、如何验证、如何展示以及发生问题后如何定位。

本次课程设计使我对 Socket 编程、HTTP 报文、正向代理、访问控制和日志审计形成了更连贯的认识。后续若继续完善,我希望进一步研究流式正文过滤、SQLite 审计存储和受控环境中的 HTTPS

内容检查，并在实现新能力时继续保留明确的安全边界和可验证证据。

参考文献

- [1] FIELDING R, NOTTINGHAM M, RESCHKE J. HTTP Semantics: RFC 9110[S]. Internet Engineering Task Force, 2022.
- [2] THOMSON M, NOTTINGHAM M. HTTP/1.1: RFC 9112[S]. Internet Engineering Task Force, 2022.
- [3] RESCORLA E. The Transport Layer Security (TLS) Protocol Version 1.3: RFC 8446[S]. Internet Engineering Task Force, 2018.
- [4] RESCHKE J. The “Basic” HTTP Authentication Scheme: RFC 7617[S]. Internet Engineering Task Force, 2015.
- [5] Python Software Foundation. socket—Low-level networking interface[EB/OL]. <https://docs.python.org/3/library/socket.html>.
- [6] Python Software Foundation. ipaddress—IPv4/IPv6 manipulation library[EB/OL]. <https://docs.python.org/3/library/ipaddress.html>.

A 关键源代码

正文重点说明功能原理与设计。本附录仅列出各核心模块最有代表性的实现片段，完整代码见首页所列项目仓库。

A.1 HTTP 请求解析、改写与转发

Listing 1 普通 HTTP 请求转发主流程

```
1 def forward_http(client_socket, request_text, request_info, config, client_ip=""):
2     """处理普通 HTTP 请求：连接目标服务器、转发请求、接收响应、执行正文过滤。"""
3     upstream_request = build_upstream_request(request_text, request_info)
4     timeout = int(config.get("timeout_seconds", DEFAULT_TIMEOUT))
5
6     try:
7         with socket.create_connection(
8             (request_info["host"], request_info["port"]),
9             timeout=timeout,
10         ) as upstream_socket:
11             upstream_socket.settimeout(timeout)
12             upstream_socket.sendall(upstream_request)
13
14             # 按 Content-Length/chunked 等 HTTP 边界接收；不能只等待 TCP 关闭，
15             # 否则 NeverSSL 等保持连接的网站会在正文已到齐后仍被误判为 504。
16             response_bytes = receive_http_response(upstream_socket, request_info["method"])
17             status_code = parse_status_code(response_bytes)
18             response_bytes, blocked_keyword = filter_response_content(
19                 response_bytes,
20                 config,
21                 request_info,
22                 client_ip,
23             )
24             if blocked_keyword:
25                 status_code = 403
26             client_socket.sendall(response_bytes)
27             return {
28                 "outcome": "filtered" if blocked_keyword else "allowed",
29                 "status_code": status_code,
30                 "bytes_sent": len(response_bytes),
31                 "keyword": blocked_keyword,
32                 "response_bytes": response_bytes,
33             }
```

A.2 访问控制策略判定

Listing 2 方法、域名与 URL 访问策略

```
1 def check_access_policy(request_info, config):
2     """执行访问控制: 方法黑名单、白名单模式、域名黑名单和 URL 关键字拦截。"""
3     host = request_info["host"]
4     method = request_info["method"]
5     path = request_info["path"]
6     target = request_info["target"]
7     mode = config.get("mode", "blacklist").lower()
8
9     blocked_methods = [item.upper() for item in config.get("blocked_methods", [])]
10    if method in blocked_methods:
11        return False, "method_blacklist"
12
13    if mode == "whitelist":
14        allowed_domains = config.get("allowed_domains", [])
15        if not domain_matches(host, allowed_domains):
16            return False, "domain_not_in_whitelist"
17
18    blocked_domains = config.get("blocked_domains", [])
19    if domain_matches(host, blocked_domains):
20        return False, "domain_blacklist"
21
22    searchable_url = f"{host}{path} {target}".lower()
23    for keyword in config.get("blocked_url_keywords", []):
24        if keyword.lower() in searchable_url:
25            return False, f"url_keyword:{keyword}"
26
27    return True, "allow"
```

A.3 HTTP 明文正文关键词过滤

Listing 3 正文类型判断、压缩解码与关键词过滤

```
1 def filter_response_content(response_bytes, config, request_info=None, client_ip=""):
2     """对 HTTP 明文文本响应做正文关键词过滤, 二进制内容直接放行。"""
3     if b"\r\n\r\n" not in response_bytes:
4         return response_bytes, None
5
6     header_bytes, body = response_bytes.split(b"\r\n\r\n", 1)
7     header_text = header_bytes.decode("iso-8859-1", errors="replace")
8     header_lines = header_text.splitlines()
9
10    content_type = response_header_value(header_lines, "Content-Type")
11    content_encoding = response_header_value(header_lines, "Content-Encoding")
12    transfer_encoding = response_header_value(header_lines, "Transfer-Encoding")
13
14    if not is_text_content_type(content_type):
15        return response_bytes, None
16
17    inspection_body = decode_chunked_body(body) if "chunked" in transfer_encoding.lower() else body
18    encoding = content_encoding.strip().lower()
19    try:
20        if encoding == "gzip":
21            inspection_body = gzip.decompress(inspection_body)
22        elif encoding == "deflate":
23            inspection_body = zlib.decompress(inspection_body)
24        elif encoding not in ("", "identity", "none"):
25            # 未支持的压缩格式保持原响应, 避免破坏客户端可用内容。
26            return response_bytes, None
```

```

27     except (OSError, zlib.error):
28         return response_bytes, None
29
30     body_text = decode_text_body(inspection_body, content_type)
31     body_text_lower = body_text.lower()
32
33     for keyword in config.get("blocked_content_keywords", []):
34         if keyword.lower() in body_text_lower:
35             return (
36                 build_policy_block_response(
37                     f"content_keyword:{keyword}",
38                     request_info,
39                     client_ip,
40                     detail=keyword,
41                 ),
42                 keyword,
43             )
44
45     return response_bytes, None

```

A.4 HTTPS CONNECT 双向隧道

Listing 4 HTTPS CONNECT 隧道双向转发

```

1 def forward_connect(client_socket, request_info, config):
2     """处理 HTTPS CONNECT: 建立 TCP 隧道, 只转发加密字节流, 不读取 HTTPS 正文。"""
3     timeout = int(config.get("timeout_seconds", DEFAULT_TIMEOUT))
4     response = b"HTTP/1.1 200 Connection Established\r\nConnection: close\r\n\r\n"
5     bytes_to_client = 0
6     bytes_from_client = 0
7
8     try:
9         with socket.create_connection(
10             (request_info["host"], request_info["port"]),
11             timeout=timeout,
12         ) as upstream_socket:
13             client_socket.sendall(response)
14             bytes_to_client += len(response)
15
16             client_socket.setblocking(False)
17             upstream_socket.setblocking(False)
18
19             sockets = [client_socket, upstream_socket]
20             last_activity = time.time()
21
22             while True:
23                 if time.time() - last_activity > timeout:
24                     break
25
26                 readable, _, exceptional = select.select(sockets, [], sockets, 1)
27                 if exceptional:
28                     break
29
30                 for ready_socket in readable:
31                     try:
32                         data = ready_socket.recv(BUFFER_SIZE)
33                     except BlockingIOError:
34                         continue
35
36                     if not data:
37                         return {
38                             "outcome": "tunnel",

```

```

39         "status_code": 200,
40         "bytes_sent": bytes_to_client,
41         "bytes_from_client": bytes_from_client,
42     }
43
44     last_activity = time.time()
45     if ready_socket is client_socket:
46         upstream_socket.sendall(data)
47         bytes_from_client += len(data)
48     else:
49         client_socket.sendall(data)
50         bytes_to_client += len(data)
51
52     return {
53         "outcome": "tunnel",
54         "status_code": 200,
55         "bytes_sent": bytes_to_client,
56         "bytes_from_client": bytes_from_client,
57     }

```

A.5 缓存、认证与固定时间窗口限流

Listing 5 GET 缓存与客户端限流桶

```

1  def set_cache(self, key, response_bytes, status_code, ttl_seconds, max_items):
2      """写入 HTTP GET 缓存，并在超过最大条目数时删除最早的缓存。"""
3      if ttl_seconds <= 0 or max_items <= 0:
4          return
5      with self.lock:
6          while max_items > 0 and len(self.cache) >= max_items:
7              oldest_key = min(
8                  self.cache,
9                  key=lambda item: self.cache[item]["created_at"],
10              )
11              del self.cache[oldest_key]
12          self.cache[key] = {
13              "response_bytes": response_bytes,
14              "status_code": status_code,
15              "created_at": time.time(),
16              "expires_at": time.time() + ttl_seconds,
17          }
18
19  def check_rate_limit(self, client_ip, max_requests, window_seconds):
20      """按客户端 IP 做固定时间窗口限流，返回是否允许和剩余/等待时间。"""
21      if max_requests <= 0:
22          return True, 0, 0
23      now = time.time()
24      with self.lock:
25          bucket = self.rate_limits.get(client_ip)
26          if not bucket or now >= bucket["reset_at"]:
27              self.rate_limits[client_ip] = {
28                  "count": 1,
29                  "reset_at": now + window_seconds,
30              }
31          return True, 0, 1
32          if bucket["count"] >= max_requests:
33              retry_after = max(1, int(bucket["reset_at"] - now))
34              return False, retry_after, bucket["count"]
35          bucket["count"] += 1
36          return True, 0, bucket["count"]

```

Listing 6 Proxy-Authorization Basic 代理认证

```
1 def check_proxy_auth(request_info, config):
2     """检查 Proxy-Authorization Basic 认证头, 认证关闭时直接放行。"""
3     if not config.get("proxy_auth_enabled", False):
4         return True
5
6     users = config.get("proxy_auth_users", {})
7     auth_header = request_info.get("headers", {}).get("proxy-authorization", "")
8     if not auth_header.lower().startswith("basic "):
9         return False
10
11     token = auth_header.split(None, 1)[1].strip()
12     try:
13         decoded = base64.b64decode(token).decode("utf-8")
14     except Exception:
15         return False
16
17     if ":" not in decoded:
18         return False
19     username, password = decoded.split(":", 1)
20     return users.get(username) == password
```

A.6 动态规则与缓存一致性

Listing 7 正文规则缓存清理与规则新增操作

```
1 def _clear_cache_for_rule_change_locked(self, rule_type, changed):
2     """正文过滤规则变化后清空旧响应缓存, 调用方必须持有 self.lock。"""
3     if changed and rule_type == "blocked_content_keywords":
4         cleared = len(self.cache)
5         self.cache.clear()
6         return cleared
7     return 0
8
9 def add_list_rule(self, rule_type, value):
10     """向指定规则列表新增一条规则, 并立即保存到配置文件。"""
11     validate_rule_type(rule_type)
12     normalized = normalize_rule_value(rule_type, value)
13     new_identity = rule_identity(rule_type, normalized)
14     with self.lock:
15         current_values = list(self.config.get(rule_type, []))
16         current_identities = {
17             rule_identity(rule_type, item)
18             for item in current_values
19         }
20         if new_identity in current_identities:
21             changed = False
22         else:
23             current_values.append(normalized)
24             self.config[rule_type] = current_values
25             changed = True
26         saved = self._save_config_locked()
27         result = {
28             "ok": True,
29             "changed": changed,
30             "saved": saved,
31             "rule_type": rule_type,
32             "value": normalized,
33             "values": list(self.config.get(rule_type, [])),
34             "cache_cleared": self._clear_cache_for_rule_change_locked(rule_type, changed),
35         }
36         result["history_entry"] = self._record_change_locked("add", result)
```

A.7 日志分类写入与结构化解析

Listing 8 日志分类写入与结构化解析

```
1 def log_event(state, event_type, message):
2     """打印并持久化事件，同时写入专用拦截或错误日志。"""
3     config = state.get_config()
4     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
5     line = f"[{timestamp}] {event_type} {message}"
6     print(line, flush=True)
7
8     write_log_line(config, "log_file", line)
9     # 认证失败和限流拒绝同样属于拦截事件。
10    if event_type.startswith(("BLOCK", "FILTER", "AUTH_REQUIRED", "RATE_LIMIT")):
11        write_log_line(config, "blocked_log_file", line)
12    if event_type.startswith("ERROR"):
13        write_log_line(config, "error_log_file", line)
14
15
16 def read_log_tail(config, kind, limit):
17     """读取最新的访问、拦截或错误日志行。"""
18     file_key = LOG_FILE_KEYS.get(kind, "log_file")
19     log_path_text = config.get(file_key)
20     if not log_path_text:
21         return []
22     log_path = resolve_project_path(log_path_text)
23     if not log_path.exists():
24         return []
25     lines = log_path.read_text(encoding="utf-8", errors="replace").splitlines()
26     return lines[-limit:]
27
28
29 def parse_log_line(line):
30     """把文本日志解析成适合查询结果表格展示的结构化字段。"""
31     match = LOG_LINE_PATTERN.match(line)
32     if not match:
33         return {"time": "", "event": "UNKNOWN", "message": line, "fields": {}, "raw": line}
34
35     message = match.group("message") or ""
36     fields = {}
37     for token in message.split():
38         if "=" not in token:
39             continue
40         key, value = token.split("=", 1)
41         fields[key] = value
42     return {
43         "time": match.group("time"),
44         "event": match.group("event"),
45         "message": message,
46         "fields": fields,
47         "raw": line,
48     }
```